

✓ Ablation: Effect of Batch Size on Contrastive Learning

Research question:

Does a larger batch size improve retrieval performance in contrastive learning?

Why batch size matters for CLIP:

The InfoNCE loss treats all other items in the batch as negatives.

A batch of 128 gives each query 127 negatives to distinguish from.

A batch of 32 gives only 31 negatives — an easier task, potentially weaker learning.

```
from google.colab import drive
drive.mount('/content/drive')

!git clone https://github.com/prajwl7676/mini-clip-DLAI.git

%cd /content/mini-clip-DLAI
!pip install -q -r requirements.txt

import sys, os
sys.path.insert(0, '/content/mini-clip-DLAI')
```

```
Mounted at /content/drive
Cloning into 'mini-clip-DLAI'...
remote: Enumerating objects: 65, done.
remote: Counting objects: 100% (65/65), done.
remote: Compressing objects: 100% (53/53), done.
remote: Total 65 (delta 24), reused 51 (delta 10), pack-reused 0 (from 0)
Receiving objects: 100% (65/65), 13.25 MiB | 16.29 MiB/s, done.
Resolving deltas: 100% (24/24), done.
/content/mini-clip-DLAI
_____ 44.4/44.4 kB 4.8 MB/s eta 0:00:00
_____ 61.0/61.0 kB 6.6 MB/s eta 0:00:00
_____ 57.6/57.6 kB 6.4 MB/s eta 0:00:00
_____ 906.4/906.4 MB 1.8 MB/s eta 0:00:00
_____ 7.2/7.2 MB 107.9 MB/s eta 0:00:00
_____ 9.7/9.7 MB 107.5 MB/s eta 0:00:00
_____ 480.6/480.6 kB 40.2 MB/s eta 0:00:00
_____ 4.5/4.5 MB 104.5 MB/s eta 0:00:00
_____ 18.0/18.0 MB 88.9 MB/s eta 0:00:00
_____ 8.3/8.3 MB 97.2 MB/s eta 0:00:00
_____ 78.3/78.3 kB 8.5 MB/s eta 0:00:00
_____ 12.9/12.9 MB 86.4 MB/s eta 0:00:00
_____ 363.4/363.4 MB 4.4 MB/s eta 0:00:00
_____ 13.8/13.8 MB 81.9 MB/s eta 0:00:00
_____ 24.6/24.6 MB 64.7 MB/s eta 0:00:00
_____ 883.7/883.7 kB 49.7 MB/s eta 0:00:00
_____ 664.8/664.8 MB 1.3 MB/s eta 0:00:00
_____ 211.5/211.5 MB 1.6 MB/s eta 0:00:00
_____ 56.3/56.3 MB 14.1 MB/s eta 0:00:00
_____ 127.9/127.9 MB 8.0 MB/s eta 0:00:00
_____ 207.5/207.5 MB 7.8 MB/s eta 0:00:00
_____ 188.7/188.7 MB 7.1 MB/s eta 0:00:00
_____ 21.1/21.1 MB 68.0 MB/s eta 0:00:00
_____ 99.1/99.1 kB 9.7 MB/s eta 0:00:00
_____ 6.2/6.2 MB 87.5 MB/s eta 0:00:00
_____ 209.6/209.6 MB 7.0 MB/s eta 0:00:00
_____ 179.3/179.3 kB 18.7 MB/s eta 0:00:00
_____ 566.4/566.4 kB 43.4 MB/s eta 0:00:00
_____ 3.1/3.1 MB 66.4 MB/s eta 0:00:00
```

ERROR: pip's dependency resolver does not currently take into account all the packages that are installed. This behaviour is the

tiFFfile 2026.4.11 requires numpy>=2.0, but you have numpy 1.26.4 which is incompatible.

jax 0.7.2 requires numpy>=2.0, but you have numpy 1.26.4 which is incompatible.

tobler 0.14.0 requires numpy>=2.0, but you have numpy 1.26.4 which is incompatible.

tobler 0.14.0 requires tqdm>=4.67, but you have tqdm 4.66.6 which is incompatible.

opencv-contrib-python 4.13.0.92 requires numpy>=2; python_version >= "3.9", but you have numpy 1.26.4 which is incompatible.

opencv-python 4.13.0.92 requires numpy>=2; python_version >= "3.9", but you have numpy 1.26.4 which is incompatible.

pytensor 2.38.3 requires numpy>=2.0, but you have numpy 1.26.4 which is incompatible.

shap 0.52.0 requires numpy>=2, but you have numpy 1.26.4 which is incompatible.

gcsfs 2025.3.0 requires fsspec==2025.3.0, but you have fsspec 2024.9.0 which is incompatible.

dataproc-spark-connect 1.1.0 requires tqdm>=4.67, but you have tqdm 4.66.6 which is incompatible.

umap-learn 0.5.12 requires scikit-learn>=1.6, but you have scikit-learn 1.5.2 which is incompatible.

xarray-einstats 0.10.0 requires numpy>=2.0, but you have numpy 1.26.4 which is incompatible.

hdbscan 0.8.44 requires scikit-learn>=1.6, but you have scikit-learn 1.5.2 which is incompatible.

opencv-python-headless 4.13.0.92 requires numpy>=2; python_version >= "3.9", but you have numpy 1.26.4 which is incompatible.

rasterio 1.5.0 requires numpy>=2, but you have numpy 1.26.4 which is incompatible.



McAfee | WebAdvisor



Your download's being scanned.
We'll let you know if there's an issue.

```
cupy-cuda12x 14.0.1 requires numpy<2.6,>=2.0, but you have numpy 1.26.4 which is incompatible.
jaxlib 0.7.2 requires numpy>=2.0, but you have numpy 1.26.4 which is incompatible.
```

```
%cd /content/mini-clip-DLAI
```

```
/content/mini-clip-DLAI
```

```
import os
import torch
import numpy as np
import matplotlib.pyplot as plt
from datasets import load_dataset
from transformers import DistilBertTokenizer

from src.model_architecture import CLIPModel
from src.dataset import FlickrDataset, get_transform, build_loaders
from src.train import train_epoch, validate, save_checkpoint, load_checkpoint
from src.evaluate import encode_dataset, recall_at_k, print_results

device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f'Device: {device}')
DRIVE_DIR = '/content/drive/MyDrive/mini-clip-dlai/ablation'
os.makedirs(DRIVE_DIR, exist_ok=True)
```

```
Device: cuda
```

Shared configuration

```
# Fixed across all runs
SHARED_CFG = {
    'total_samples': 10_000,
    'n_train': 8_000,
    'n_val': 1_000,
    'embedding_dim': 256,
    'num_epochs': 8,
    'lr': 1e-4,
    'lr_head': 1e-3,
    'weight_decay': 1e-2,
    'max_grad_norm': 1.0,
    'num_workers': 2,
}

# The variable we are ablating
BATCH_SIZES = [32, 64, 128]

print('Shared config:')
for k, v in SHARED_CFG.items():
    print(f' {k:<20}: {v}')
print(f'\nBatch sizes to test: {BATCH_SIZES}')
```

```
Shared config:
total_samples      : 10000
n_train            : 8000
n_val              : 1000
embedding_dim      : 256
num_epochs         : 8
lr                 : 0.0001
lr_head            : 0.001
weight_decay       : 0.01
max_grad_norm      : 1.0
num_workers        : 2

Batch sizes to test: [32, 64, 128]
```

Load data

```
full_ds = load_dataset('nlphuji/flickr30k', split='test')
tokenizer = DistilBertTokenizer.from_pretrained('distilbert-base-uncased')

n_train = SHARED_CFG['n_train']
```



McAfee | WebAdvisor



Your download's being scanned.
We'll let you know if there's an issue.

```

n_val = SHARED_CFG['n_val']

train_hf = full_ds.select(range(0, n_train))
val_hf = full_ds.select(range(n_train, n_train + n_val))
test_hf = full_ds.select(range(n_train + n_val, SHARED_CFG['total_samples']))

train_ds = FlickrDataset(train_hf, tokenizer, get_transform(train=True))
val_ds = FlickrDataset(val_hf, tokenizer, get_transform(train=False))
test_ds = FlickrDataset(test_hf, tokenizer, get_transform(train=False))

print(f'Train: {len(train_ds):,} | Val: {len(val_ds):,} | Test: {len(test_ds):,}')

```

/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:

The secret `HF\_TOKEN` does not exist in your Colab secrets.

To authenticate with the Hugging Face Hub, create a token in your settings tab (<https://huggingface.co/settings/tokens>), set it

You will be able to reuse this secret in all of your notebooks.

Please note that authentication is recommended but still optional to access public models or datasets.

warnings.warn(

README.md: 100% 641/641 [00:00<00:00, 57.9kB/s]

flickr30k.py: 2.51k/? [00:00<00:00, 218kB/s]

TEST/test/0000.parquet: 100% 506M/506M [00:04<00:00, 201MB/s]

TEST/test/0001.parquet: 100% 502M/502M [00:05<00:00, 106MB/s]

TEST/test/0002.parquet: 100% 506M/506M [00:05<00:00, 90.2MB/s]

TEST/test/0003.parquet: 100% 512M/512M [00:03<00:00, 171MB/s]

TEST/test/0004.parquet: 100% 504M/504M [00:03<00:00, 179MB/s]

TEST/test/0005.parquet: 100% 495M/495M [00:03<00:00, 169MB/s]

TEST/test/0006.parquet: 100% 495M/495M [00:06<00:00, 69.2MB/s]

TEST/test/0007.parquet: 100% 497M/497M [00:08<00:00, 64.6MB/s]

TEST/test/0008.parquet: 100% 289M/289M [00:04<00:00, 74.7MB/s]

Generating test split: 100% 31014/31014 [00:50<00:00, 184.49 examples/s]

tokenizer_config.json: 100% 48.0/48.0 [00:00<00:00, 2.70kB/s]

vocab.txt: 232k/? [00:00<00:00, 2.63MB/s]

tokenizer.json: 466k/? [00:00<00:00, 4.64MB/s]

config.json: 100% 483/483 [00:00<00:00, 42.7kB/s]

Train: 8,000 | Val: 1,000 | Test: 1,000

✓ Training function for one ablation run

```

import math
def run_ablation(batch_size: int, cfg: dict) -> dict:
    """
    Train a fresh CLIPModel with the given batch_size.
    Returns the final val loss and test Recall@K results.
    """
    print(f'\n{"+" * 60}')
    print(f' ABLATION RUN - batch_size={batch_size}')
    print(f' Random loss baseline: log({batch_size}) = {math.log(batch_size):.3f}')
    print(f'\n{"+" * 60}')

    # — DataLoaders —————
    train_loader, val_loader, test_loader = build_loaders(
        train_ds, val_ds, test_ds,
        batch_size = batch_size,
        num_workers = cfg['num_workers'],
    )

    # — Fresh model —————
    # IMPORTANT: fresh model each run — same random seed for fairness
    torch.manual_seed(42)
    model = CLIPModel(embedding_dim=cfg['embedding_dim']).to(device)

    # — Optimizer + scheduler —————

```



McAfee | WebAdvisor



Your download's being scanned.
We'll let you know if there's an issue.

```

projection_params = (
    list(model.image_encoder.projection.parameters()) +
    list(model.text_encoder.projection.parameters()) +
    [model.log_temperature]
)
encoder_params = [
    p for p in model.parameters()
    if p.requires_grad and not any(p is pp for pp in projection_params)
]
optimizer = torch.optim.AdamW([
    {'params': encoder_params, 'lr': cfg['lr'], 'weight_decay': cfg['weight_decay']},
    {'params': projection_params, 'lr': cfg['lr_head'], 'weight_decay': 0.0},
])
scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(
    optimizer, T_max=cfg['num_epochs'], eta_min=1e-6
)

# — Training loop —————
loss_history = {'train': [], 'val': []}
best_val_loss = float('inf')
best_ckpt = os.path.join(DRIVE_DIR, f'best_bs{batch_size}.pt')

for epoch in range(cfg['num_epochs']):
    tr = train_epoch(model, train_loader, optimizer, device, cfg['max_grad_norm'])
    vl = validate(model, val_loader, device)
    scheduler.step()

    loss_history['train'].append(tr['loss'])
    loss_history['val'].append(vl['loss'])

    improved = ' ← best' if vl['loss'] < best_val_loss else ''
    print(f' Epoch {epoch+1:02d}/{cfg["num_epochs"]:02d} | '
          f'train {tr["loss"]:.4f} | val {vl["loss"]:.4f}{improved}')

    if vl['loss'] < best_val_loss:
        best_val_loss = vl['loss']
        save_checkpoint(model, optimizer, epoch,
                        tr['loss'], vl['loss'], loss_history,
                        save_path=best_ckpt)

# — Load best and evaluate —————
ckpt = load_checkpoint(model, optimizer, best_ckpt, device)
embeddings = encode_dataset(model, test_loader, device)
results = recall_at_k(
    embeddings['image_embeddings'],
    embeddings['text_embeddings'],
    ks=[1, 5, 10]
)

print_results(results, title=f' batch_size={batch_size}')

return {
    'batch_size': batch_size,
    'best_val_loss': best_val_loss,
    'results': results,
    'loss_history': loss_history,
}

print('run_ablation() defined.')

run_ablation() defined.

```

▼ three ablation experiments

```

ablation_results = []

for bs in BATCH_SIZES:
    run_result = run_ablation(bs, SHARED_CFG)
    ablation_results.append(run_result)

print('\nAll ablation runs complete.')

```



McAfee | WebAdvisor



Your download's being scanned.
We'll let you know if there's an issue.



McAfee | WebAdvisor



Your download's being scanned.
We'll let you know if there's an issue.

```

+++++
ABLATION RUN - batch_size=32
Random loss baseline: log(32) = 3.466
+++++
Downloading: "https://download.pytorch.org/models/resnet50-0676ba61.pth" to /root/.cache/torch/hub/checkpoints/resnet50-0676ba61
100%|██████████| 97.8M/97.8M [00:00<00:00, 163MB/s]

```

✓ Ablation results table

```

model.safetensors: 100%
268M/268M [00:02<00:00, 120MB/s]
Epoch 01/08 | train 1.7631 | val 1.3874 ← best
Epoch 02/08 | train 1.0380 | val 1.2473 ← best

print('\n=== Ablation Results: Effect of Batch Size ===')
print(f'{"Batch size":<12} {"t2i R@1":>9} {"t2i R@5":>9} {"t2i R@10":>10} {"i2t R@1":>9} {"Val Loss":>10}')
print('-' * 62)

for r in ablation_results:
    bs = r['batch_size']
    res = r['results']
    vl = r['best_val_loss']
    print(
        f'{bs:<12} '
        f'{res["t2i_R@1"]:>8.2f}% '
        f'{res["t2i_R@5"]:>8.2f}% '
        f'{res["t2i_R@10"]:>9.2f}% '
        f'{res["i2t_R@1"]:>8.2f}% '
        f'{vl:>10.4f}'
    )
print('-' * 62)

```

```

R@10    61.50% ██████████
=== Ablation Results: Effect of Batch Size ===
Batch size    t2i R@1    t2i R@5    t2i R@10    i2t R@1    Val Loss
+++++
32ABLATION RUN - batch_size=32    59.30%    18.60%    1.1798
64Random loss baseline: log(64) = 4.1817
128+++++
Epoch 01/08 | train 2.2580 | val 1.8177 ← best
Epoch 02/08 | train 1.3108 | val 1.5859 ← best
Epoch 03/08 | train 0.8693 | val 1.5420 ← best
Epoch 04/08 | train 0.5797 | val 1.5356 ← best
Epoch 05/08 | train 0.3723 | val 1.4874 ← best
Epoch 06/08 | train 0.2559 | val 1.4792 ← best
Epoch 07/08 | train 0.1838 | val 1.4761 ← best
Epoch 08/08 | train 0.1530 | val 1.4730 ← best

```

✓ Plot ablation results

```

batch_sizes = [r['batch_size'] for r in ablation_results]
t2i_r1 = [r['results']['t2i_R@1'] for r in ablation_results]
t2i_r5 = [r['results']['t2i_R@5'] for r in ablation_results]
i2t_r1 = [r['results']['i2t_R@1'] for r in ablation_results]
val_losses = [r['best_val_loss'] for r in ablation_results]

fig, axes = plt.subplots(1, 2, figsize=(12, 5))

# Left: Recall@K vs batch size
ax = axes[0]
ax.plot(batch_sizes, t2i_r1, 'o-', color='steelblue', lw=2, ms=8, label='t2i R@1')
ax.plot(batch_sizes, t2i_r5, 's-', color='mediumseagreen', lw=2, ms=8, label='t2i R@5')
ax.plot(batch_sizes, i2t_r1, '^-', color='tomato', lw=2, ms=8, label='i2t R@1')
ax.set_xlabel('Batch size')
ax.set_ylabel('Recall (%)')
ax.set_title('Retrieval Performance vs Batch Size')
ax.set_xticks(batch_sizes)
ax.legend()
ax.grid(True, alpha=0.3)

# Right: Val loss vs batch size
ax2 = axes[1]
bars = ax2.bar([str(bs) for bs in batch_sizes], val_losses,
               color=['#B0BEC5', '#1D9E75', '#7F77DD'], edgecolor='white', width=0.5)
ax2.set_xlabel('Batch size')
ax2.set_ylabel('Best Validation Loss')
ax2.set_title('Validation Loss vs Batch Size')
ax2.grid(True, axis='y', alpha=0.3)

for bar, vl in zip(bars, val_losses):
    ax2.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.01,
             f'{vl:.4f}', ha='center', va='bottom', fontsize=9)

# Add log(B) baseline markers
for bs in batch_sizes:

```



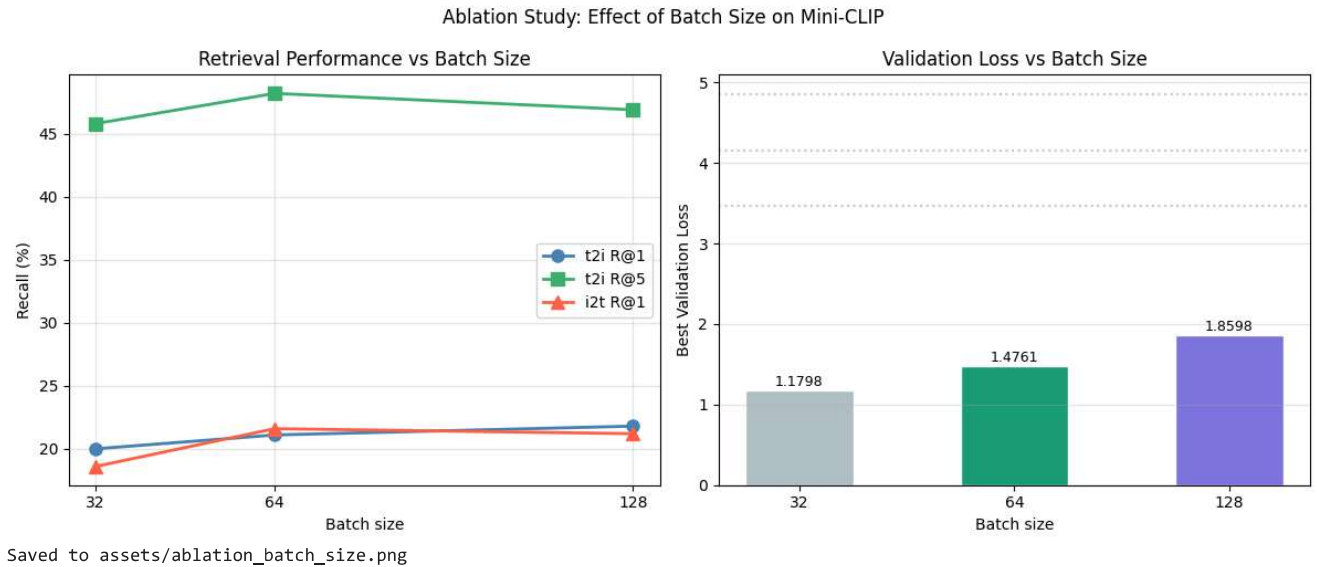
McAfee | WebAdvisor



Your download's being scanned.
We'll let you know if there's an issue.

```
ax2.axhline(math.log(bs), linestyle=':', color='grey', alpha=0.4)

plt.suptitle('Ablation Study: Effect of Batch Size on Mini-CLIP', fontsize=12)
plt.tight_layout()
plt.savefig('assets/ablation_batch_size.png', dpi=120, bbox_inches='tight')
plt.show()
print('Saved to assets/ablation_batch_size.png')
```



Close

```
---
## Loss curves for all three runs
```

Loss curves for all three runs

```
fig, axes = plt.subplots(1, 3, figsize=(15, 4), sharey=True)
colors = ['#B0BEC5', '#1D9E75', '#7F77DD']

for ax, r, color in zip(axes, ablation_results, colors):
    bs = r['batch_size']
    history = r['loss_history']
    epochs = range(1, len(history['train']) + 1)

    ax.plot(epochs, history['train'], 'o-', color=color, lw=2, ms=4, label='train')
    ax.plot(epochs, history['val'], 's--', color=color, lw=2, ms=4, alpha=0.6, label='val')
    ax.axhline(math.log(bs), color='grey', linestyle=':', lw=1,
               label=f'log({bs})={math.log(bs):.2f}')
    ax.set_title(f'batch_size = {bs}')
    ax.set_xlabel('Epoch')
    ax.set_ylabel('InfoNCE Loss')
    ax.legend(fontsize=8)
    ax.grid(True, alpha=0.3)

plt.suptitle('Training curves by batch size', fontsize=12)
plt.tight_layout()
plt.savefig('assets/ablation_loss_curves.png', dpi=120, bbox_inches='tight')
plt.show()
```



McAfee | WebAdvisor



Your download's being scanned.
We'll let you know if there's an issue.